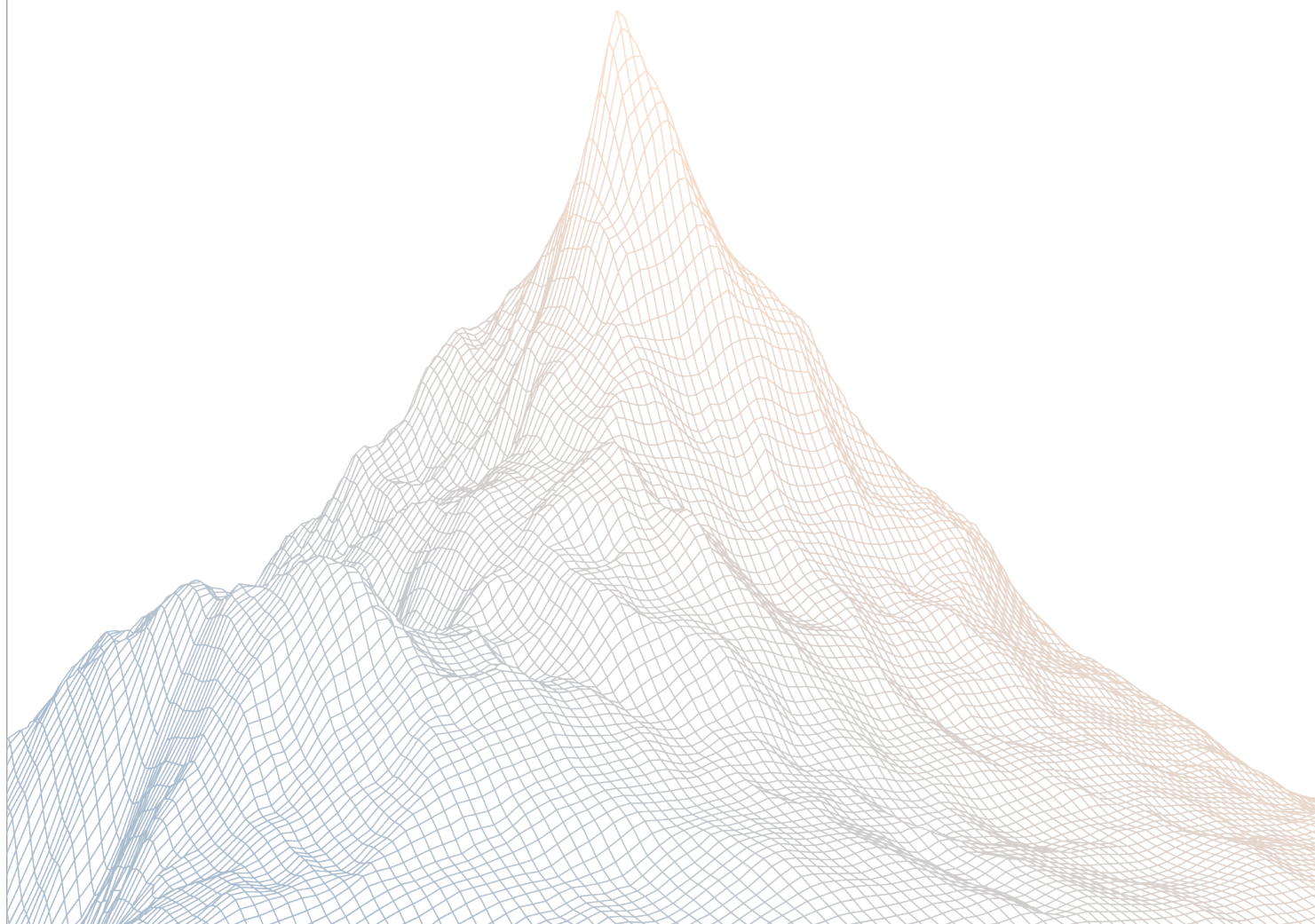


Techdollar

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

July 24th to July 30th, 2026

AUDITED BY:

Matte
ether_sky

Contents

1	Introduction	3
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	4
2.1	About Techdollar	4
2.2	Scope	4
2.3	Audit Timeline	5
2.4	Issues Found	5
3	Findings Summary	6
4	Findings	7
4.1	Low Risk	7
4.2	Informational	18

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Techdollar

Providing private equity shareholders with liquidity for quality of life improvements, while preserving their upside

2.2 Scope

The engagement involved a review of the following targets:

Target	contracts
---------------	-----------

Repository	https://github.com/techdollar/contracts
-------------------	---

Commit Hash	be198fe9ce58ba5fc5edd09b9742d7de3e313835
--------------------	--

Files	src/**/*.sol
--------------	--------------

Target	Mitigation Review
---------------	-------------------

Repository	https://github.com/techdollar/contracts
-------------------	---

Commit Hash	2421761267a9c097528767afafeb86f8f4ffc584
--------------------	--

Files	src/**/*.sol
--------------	--------------

2.3 Audit Timeline

July 24, 2026	Audit start
July 30, 2026	Audit end
August 20, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	9
Informational	6
Total Issues	15

3

Findings Summary

ID	Description	Status
L-1	repayOnchainLoanPrincipal requires a collector for zero fees	Resolved
L-2	The interest can be paid even when only dead shares exist	Resolved
L-3	Fiat and on-chain loans use the same verification process	Resolved
L-4	The staking rate should be rounded up	Resolved
L-5	LoanVerifier oracle can freeze an active on-chain loan by removing its attestation	Resolved
L-6	initiateOnchainLoan permits fee-bearing loans without a collector	Resolved
L-7	kyberRouter no-code targets accept zero-output redemptions	Acknowledged
L-8	Mutable loanVerifier state freezes accepted-loan settlement	Resolved
L-9	Pause policy asymmetry preserves owner liquidation	Acknowledged
I-1	The available balance should be checked when requesting a withdrawals	Acknowledged
I-2	setUSDte breaks the cached collateral invariant	Resolved
I-3	previewStake ignores the bootstrap share deduction	Acknowledged
I-4	Use the same address for native token in swap functions	Acknowledged
I-5	Cancelling withdrawals is possible when the contract is paused	Resolved
I-6	The repayment and liquidation could be possible simultaneously after maturity	Acknowledged

4

Findings

4.1 Low Risk

A total of 9 low risk findings were identified.

[L-1] `repayOnchainLoanPrincipal` requires a collector for zero fees

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [eUSDte.sol](#)

Description

A zero-fee loan should close without a fee destination because no fee transfer occurs. I traced the mature principal path: `exitFee` is computed as zero, yet `repayOnchainLoanPrincipal()` still rejects a zero `feeCollector` before assigning `Closed` status.

```
if (interestShortfall > 0) {
    loan.totalPaid += interestShortfall;
    totalInterestPaid += interestShortfall;
}

if (feeCollector == address(0)) revert InvalidAddress();
if (exitFee > 0) {
    usdte.safeTransfer(feeCollector, exitFee);
}
```

For a 1,024-unit principal and `exitFeeBps = 0`, the calculation is $1,024 * 0 = 0$, but the unrelated address guard still reverts. The EVM unwinds the preceding token transfer and accounting writes; liveness is lost until the owner supplies an economically unused address.

Recommendation

Validate `feeCollector` only inside the positive-fee branch. This also repairs already-created zero-fee loans without weakening fee-bearing settlement.

TechDollar: Resolved with [@21859a74e8 ...](#)

Zenith: Verified.

[L-2] The interest can be paid even when only dead shares exist

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [eUSDte.sol](#)

Description:

To avoid paying interest when there are no stakers, the checks below were added.

```
function payInterest(  
    uint256 amount,  
    bytes32 loanId  
) external onlyOwner nonReentrant {  
    if (totalSupply() == 0) revert NoStakersToReceiveInterest();
```

However, during the first staking operation, dead shares are minted to the dead address, and these shares will exist permanently.

Recommendations:

The check below would be more accurate.

```
function payInterest(  
    uint256 amount,  
    bytes32 loanId  
) external onlyOwner nonReentrant {  
    if (totalSupply() == 0) revert NoStakersToReceiveInterest();  
    if (totalSupply() ≤ DEAD_SHARES) revert NoStakersToReceiveInterest();
```

TechDollar: Resolved with [@a88b348e7f...](#)

Zenith: Verified.

[L-3] Fiat and on-chain loans use the same verification process

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [eUSDte.sol](#)

Description:

Currently, fiat and on-chain loans use the same verification process.

```
function initiateFiatLoan(
    uint256 amount,
    address receiver,
    bytes32 loanId
) external onlyOwner nonReentrant {
    _verifyLoan(receiver, amount, loanId);

    function initiateOnchainLoan(
        uint256 amount,
        address borrower,
        uint256 rateBps,
        uint256 numberOfPayments,
        uint256 maturity,
        bytes32 loanId,
        uint256 exitFeeBps
    ) external onlyOwner nonReentrant {
        _verifyLoan(borrower, amount, loanId);
```

This means that if the oracle approves a loan, the owner can create both fiat and on-chain loans using the same verification.

Recommendations:

Use a specific verification process for each loan type.

TechDollar: Resolved with [@c3edf2462e ...](#)

Zenith: Verified.

[L-4] The staking rate should be rounded up

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: High

Target

- [eUSDte.sol](#)

Description:

The exchange rate is always rounded down and is used by the stake function.

```
function exchangeRate() public view returns (uint256 rate) {
    uint256 supply = totalSupply();
    if (supply == 0) {
        return PRECISION; // Initial rate 1:1
    }

    uint256 backing = totalBacking();
    rate = (backing * PRECISION) / supply;

    if (rate == 0) revert ExchangeRateCollapsed();

    return rate;
}

function stake(
    uint256 usdteAmount,
    uint256 minSharesOut
) external nonReentrant whenNotPaused returns (uint256 eUSDteMinted) {
    if (usdteAmount == 0) revert InvalidAmount();
    if (minSharesOut == 0) revert InvalidAmount();

    uint256 rate = exchangeRate();
    eUSDteMinted = (usdteAmount * PRECISION) / rate;
}
```

Since the minted shares should be rounded down when staking assets, the exchange rate in this calculation should be rounded up, as is done in the ERC-4626 vault implementation.

Recommendations:

The exchange rate should be rounded up when staking and rounded down when withdrawing.

TechDollar: Resolved with [@efad041fdc...](#)

Zenith: Verified.

[L-5] LoanVerifier oracle can freeze an active on-chain loan by removing its attestation

SEVERITY: Low

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Low

Target

- [eUSDte.sol](#)

Description:

Every loan operation, including a borrower's repayment and the owner's liquidation, first requires the loan to still be attested:

```
function _verifyLoan(address borrower, uint256 amount, bytes32 loanId)
    internal view {
        if (address(loanVerifier) == address(0)) revert VerifierNotSet();
        if (!loanVerifier.isVerified(borrower, amount, loanId))
            revert LoanNotVerified();
    }
```

The oracle can un-attest a loan at any time. If the oracle removes the attestation for a live loan, then `payInterest`, `repayOnchainLoanInterest`, `repayOnchainLoanPrincipal`, and `liquidateOnchainLoan` all revert with `LoanNotVerified`. The borrower can no longer pay down or close their loan, and the owner cannot liquidate it. The loan remains frozen until it is attested. If the loan's borrower is not the zero address, it means the loan has already been verified during its creation.

Recommendations:

Remove loan verification from the functions above.

TechDollar: Resolved with [@b35761c6e0...](#)

Zenith: Verified.

[L-6] `initiateOnchainLoan` permits fee-bearing loans without a collector

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [eUSDte.sol](#)

Description

Every fee-bearing on-chain loan should have a usable exit-fee destination before principal leaves the vault. I traced `initiateOnchainLoan()` into `repayOnchainLoanPrincipal()`: origination accepts a positive `exitFeeBps` while `feeCollector` is still zero, but settlement later rejects that same configuration.

```
if (exitFeeBps > BPS_DENOMINATOR) revert ExitFeeTooHigh();
if (onchainLoans[loanId].borrower != address(0)) {
    revert OnchainLoanAlreadyExists();
}
_verifyLoan(borrower, amount, loanId);

uint256 paymentPeriod = maturity / numberOfPayments;
```

A loan with a 100-bps fee and a 1,024-unit principal computes $10 > 0$ fee units, yet closure reverts until the owner calls `setFeeCollector()`. The failure is atomic. Principal and borrower balances roll back, but the accepted position remains Active and unsettled.

Recommendation

Require a nonzero collector before originating any loan with a positive exit fee. Keep the invariant explicit at origination so settlement never depends on later owner configuration.

TechDollar: Resolved with [@21859a74e8 ...](#)

Zenith: Verified.

[L-7] kyberRouter no-code targets accept zero-output redemptions

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [USDteMintHelper.sol](#)

Description

A successful redemption swap should prove that the configured router executed code and produced assets. I followed `_redeemAndSwap()` after `usdte.redeem()`: a low-level call to an EOA reports success, while a caller-selected zero floor lets the function finish with no output and leaves AUDS in the helper (the owner can recover it).

```
uint256 amountOut;
if (tokenOut == address(0)) {
    uint256 nativeBefore = address(this).balance;
    (bool success, ) = kyberRouter.call(swapData);
    if (!success) revert SwapFailed();

    amountOut = address(this).balance - nativeBefore;
```

With `minAmountOut = 0`, an observed output of `0 = 0` passes; changing the floor to 1 makes `0 < 1` and reverts the entire redemption. The path therefore needs both an owner router mistake and a user who disables the existing delta check.

Recommendation

Reject router addresses without deployed code and require a positive output floor for value-bearing routes. Preserve the balance-delta check as the final execution proof.

TechDollar: Acknowledged. We are not applying a code change for this finding. kyberRouter is set once, at initialization, from a reviewed constant in our deployment script.

[L-8] Mutable loanVerifier state freezes accepted-loan settlement

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [eUSDte.sol](#)

Description

An attestation accepted at origination should not disappear as a prerequisite for settling the resulting debt. I traced `removeLoan()` through `_verifyLoan()`: interest payment, mature principal repayment, and liquidation all read the mutable live predicate instead of the loan's accepted state.

```
function _verifyLoan(  
    address borrower,  
    uint256 amount,  
    bytes32 loanId  
) internal view {  
    if (address(loanVerifier) == address(0)) revert VerifierNotSet();  
    if (!loanVerifier.isVerified(borrower, amount, loanId))  
        revert LoanNotVerified();  
}
```

After one registered 1,024-unit loan becomes Active, toggling its verifier entry from 1 to 0 makes all three settlement paths revert; restoring 0 + 1 = 1 permits closure again. Recovery is privileged and straightforward, but pooled principal remains outstanding during the freeze.

Recommendation

Persist accepted authorization at origination or define a revocation state that still permits repayment and enforcement. Do not make mutable admission evidence a continuing settlement dependency.

TechDollar: Resolved with [@b35761c6e0 ...](#)

Zenith: Verified.

[L-9] Pause policy asymmetry preserves owner liquidation

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [eUSDte.sol](#)

Description

A paused loan lifecycle should not let one side finalize a position while the counterparty's cure paths are disabled. I followed `pause()` into the on-chain loan functions: both borrower repayment entry points use `whenNotPaused`, while `liquidateOnchainLoan()` and `coverLiquidatedLoan()` omit it.

```
function liquidateOnchainLoan(
    bytes32 loanId
) external onlyOwner nonReentrant {
    OnchainLoan storage loan = onchainLoans[loanId];
    if (loan.borrower == address(0)) revert OnchainLoanNotFound();
    if (loan.status != OnchainLoanStatus.Active)
        revert OnchainLoanNotActive();
    _verifyLoan(loan.borrower, loan.originalPrincipal, loanId);
}
```

During one pause, borrower access changes from 1 to 0 while owner liquidation remains 1 = 1. Eligibility still requires delinquency or maturity, and liquidation transfers no borrower collateral, but the status transition permanently removes the on-chain repayment route.

Recommendation

Apply one documented pause policy to repayment, liquidation, and coverage. If borrower cures are disabled, block the owner state transitions that would make that lockout permanent.

TechDollar: Acknowledged. We are aware of the asymmetry and are keeping the current behaviour.

4.2 Informational

A total of 6 informational findings were identified.

[I-1] The available balance should be checked when requesting a withdrawals

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [eUSDte.sol](#)

Description:

The `available balance` is not checked in the `requestWithdrawal` function. As a result, users can create `withdrawal` requests even when there are insufficient tokens available. This can cause a situation where later users are able to withdraw their tokens, while earlier users cannot complete their withdrawals if they did not call the `withdraw` function at the appropriate time.

Recommendations:

Check the `available balance` in the `requestWithdrawal` function.

TechDollar: Acknowledged. We will not be fixing this the behavior is intentional : Checking the available balance at request time would reject legitimate withdrawal requests whenever funds are temporarily deployed in loans, which is the normal operating state of the vault. The 7-day cooldown is designed to give the owner time to replenish liquidity before the withdrawal becomes executable. We prefer that a user's request is always accepted and waits in the queue until liquidity is restored, rather than reverting at request time and forcing the user to retry. Shares are burned at the exchange rate observed at request time, so the queued amount is fixed and cannot be diluted while waiting.

[I-2] setUSDte breaks the cached collateral invariant

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [USDteMintHelper.sol](#)

Description

The helper's cached AUSD must match the collateral exposed by its active usdte contract. I traced initialization into `setUSDte()`: initialization reads both references together, while replacement changes only `usdte`, allowing an owner-selected, differently backed deployment to split the pair.

```
function setUSDte(address _usdte) external onlyOwner {
    if (_usdte == address(0)) revert InvalidAddress();

    address oldUSDte = address(usdte);
    usdte = IUSDte(_usdte);

    emit USDteUpdated(oldUSDte, _usdte);
}
```

After one incompatible rotation, the invariant count becomes $1 \neq 2$: balance deltas and approvals use the old token while mint or redeem uses the new token. Focused calls revert atomically, so users retain assets, and the owner can restore a same-AUSD USDte.

Recommendation

Validate that a replacement returns the cached AUSD address, or update both references atomically as an explicit collateral migration. Pause the helper during any supported migration.

TechDollar: Resolved with [@8a8d583d3e ...](#)

Zenith: Verified.

[I-3] previewStake ignores the bootstrap share deduction

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [eUSDte.sol](#)

Description

The empty-vault preview should return the same share amount that stake() can mint to its caller. I traced the bootstrap branch: stake() deducts 1,000 dead shares from the first user's quote, while previewStake() returns the unadjusted calculation and exposes no minimum-deposit boundary.

```
function previewStake(  
    uint256 usdteAmount  
) external view returns (uint256 eUSDteAmount) {  
    uint256 rate = exchangeRate();  
    return (usdteAmount * PRECISION) / rate;  
}
```

A first deposit quoted at 1,024 base units yields only $1,024 - 1,000 = 24$ user shares; using the quote as minSharesOut reverts. Deposits below 1,000 underflow, while exactly 1,000 produces zero before failing the positive minimum. The intended permanent lock is only 0.001 USDte.

Recommendation

Mirror the bootstrap deduction in previewStake() and reject first deposits that cannot absorb DEAD_SHARES.

TechDollar: Acknowledged. We don't use this function for estimate amount out, this is only for info on UI.

[I-4] Use the same address for native token in swap functions

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [USDteMintHelper.sol](#)

Description:

Some functions use `address(0)` for native token.

```
function mintWithSwap(
    address tokenIn,
    uint256 amountIn,
    uint256 minAusdOut,
    bytes calldata swapData
) external payable nonReentrant whenNotPaused {
    if (tokenIn == address(0)) {
```

Other functions use `NATIVE_TOKEN`.

```
function swap(
    address tokenIn,
    address tokenOut,
    uint256 amountIn,
    uint256 minAmountOut,
    bytes calldata swapData
) external payable nonReentrant whenNotPaused {
    if (amountIn == 0) revert InvalidAmount();
    if (tokenIn == tokenOut) revert SameToken();

    // Pull input
    if (tokenIn == NATIVE_TOKEN) {
```

Recommendations:

Use same address for all functions.

TechDollar: Acknowledged.

[I-5] Cancelling withdrawals is possible when the contract is paused

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- [eUSDte.sol](#)

Description:

There is no whenNotPaused modifier in the cancelWithdrawal function.

```
function cancelWithdrawal(uint256 requestId) external nonReentrant {
```

Recommendations:

```
function cancelWithdrawal(uint256 requestId) external nonReentrant {  
function cancelWithdrawal(uint256 requestId)  
external nonReentrant whenNotPaused {
```

TechDollar: Resolved with [@026fecdb6c ...](#)

Zenith: Verified.

[I-6] The repayment and liquidation could be possible simultaneously after maturity

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [eUSDte.sol](#)

Description:

After maturity in on-chain loan, the repayment and liquidation could be possible simultaneously.

Recommendations:

This could be documented.

TechDollar: Acknowledged. Liquidation is driven by our backend, which applies a 7-day grace period after maturity.